WebGraph.metehan.ai - Analysis, Rebuild Blueprint and Budget

Prepared as a hands-on technical and competitive teardown, written for a website owner who wants to build something similar in-house. July 2026.

1. What This Tool Actually Is, In Plain English

Imagine every website on the internet as a guest at a giant networking event, and every hyperlink from one site to another as a handshake or a personal introduction. If thousands of well-respected people keep introducing you to others, you naturally become someone people trust. WebGraph.metehan.ai is a **free lookup tool** that tells you, for any domain, roughly how many of those introductions it has received, who is doing the introducing, and how that reputation has shifted over the last three years. It does not crawl the web itself in real time. Instead, it reads a free, publicly released map of the web called the **Common Crawl**

WebGraph and turns that raw map into simple **scores**, **rankings**, and **lists** that anyone can search in a browser.

2. The Two Tools Bundled Inside the Site

There are really two connected mini-apps living under one homepage.

2.1 CC Rank Checker (the homepage tool)

You type in one or more domains (for example peec.ai, tesla.com, wikipedia.org) and get back, for each one: its **Harmonic Centrality rank (called HC Rank),** its **PageRank**, its subdomain count, a tier badge such as Elite, Top 1K or Top 100K, and a small line chart showing how both ranks moved across twelve historical snapshots stretching from 2023 to mid-2026. There is also a public Top 1000 **leaderboard page** listing the **highest-ranked domains** on the entire web graph, topped by infrastructure giants like googleapis.com, facebook.com and google.com.

2.2 Referring Domains Checker (the deeper tool)

This second tool, embedded from a companion Cloudflare Worker, is where the real analysis happens. Enter a single root domain and it returns an Authority Score and a Citation Score (both rescaled to an easy 0-100 range), a full list of every distinct domain that links to you, a bar chart showing how many of those referring domains are weak versus strong, and a breakdown of which top-level domains (.com, .io, .ai, and so on) send you the most links. It also draws a **small radial network graph** showing your site in the center connected to its **top referring domains**. A separate **Link Gap Finder** tab lets you compare your own domain against one or more competitors and returns every domain that links to a competitor but not to you, then flags the subset that is realistically attainable (linking to more than one competitor, mid-tier authority, not an impossible mega-site like Google or Wikipedia). A third tab, the Cheatsheet, gives plain-language tips such as prioritizing links from a wide variety of domains rather than many links from one site, and treating

high-outbound-link domains as directories or guest-post opportunities rather than one-off link requests

3. The Core Idea: Two Different Ways of Scoring Importance

PageRank, invented by Google's founders, treats every hyperlink as a vote. A link from a small, obscure blog counts as a small vote. A link from a domain that itself has thousands of trusted incoming links counts as a much bigger vote, because that domain is effectively lending you some of its own credibility. This is why one link from a well-known industry publication can matter more than fifty links from unknown directories.

Harmonic Centrality measures something subtly different: **how short the average path is from every other domain on the web to your domain, following links**. Picture the whole web as a **subway map** where a link is a rail line between two stations.

A domain with a low Harmonic Centrality rank (which the tool reports as a better, more central position) is like a station a

few stops away from almost everywhere else in the city. It does not need one enormous vote of confidence; it just needs to sit in a well-connected part of the network. In practice, HC Rank rewards being widely and diversely linked, while PageRank rewards being linked by a few extremely powerful sites, and the two rankings frequently disagree slightly on the same domain.

3.1 Complete List of Every Logic and Algorithm the Tool Uses

The full document above explains the two headline scoring systems in depth. Here is every distinct piece of logic the tool runs, consolidated in one place:

- Data foundation: uses Common Crawl's free, public Host/Domain-level Web Graph instead of running its own crawler — three monthly releases merged into one graph.
- Domain-level graph construction: collapses page-level links into domain-to-domain edges (about 121 million nodes, 3.9 billion edges), so it tracks referring domains, not raw URLs.

- Harmonic Centrality algorithm: computes the average inverse shortest-path distance from every other domain to a given domain, rewarding domains that sit centrally in the link graph. Displayed as HC Rank / Authority Score.
- PageRank algorithm: the classic recursive vote-weighting model, where links from highly-linked domains count more. Displayed as PageRank Rank / Citation Score.
- Score normalization: raw graph-theory values (tiny floating point numbers) are converted into readable percentile ranks and 0-100 style scores.
- Referring-domains counting: counts distinct linking domains rather than total backlinks, so one domain linking 500 times still counts once.
- Outbound-domains logic: the same edge list read in reverse, used to build prospecting lists of domains a site already trusts enough to link out to.

- TLD distribution logic: groups referring domains by top-level domain (.com, .ai, .io, etc.) to show link-profile composition at a glance.
- Network graph rendering: a visualization layer plotting referring-domain relationships radially; uses the same edge data as the other metrics, just displayed differently.
- Link Gap Finder (set-difference logic): computes domains that link to one or more competitor domains but not to your domain, producing a raw gap list.
- Relevance/attainability filter: applies heuristics on top of the raw gap list to flag a smaller subset as niche-relevant and realistically attainable.
- Platform-opportunity detection: separately flags very-high-outbound-link domains (like github.io) as open platforms worth listing on, versus one-off editorial link targets.
- Serving architecture: pre-computed results only (nothing scraped live) - Cloudflare KV plus a

Cloudflare Worker for instant CC Rank Checker lookups, and a thin Worker proxying a Hugging Face Space backend for the heavier Referring Domains / Link Gap logic.

- Public API layer: both tools expose plain JSON endpoints (/api/report and /api/gap) behind the scenes, so the same logic can be scripted or embedded without the front-end.
- Freshness/update-cadence logic: tied to Common Crawl's own release schedule rather than continuous crawling, so brand-new pages or links can take time to appear.

4. Internal Architecture: How It Is Actually Built

The entire system is built on one free, public dataset rather than a private crawler. Common Crawl, a non-profit, releases a Host and Domain-level Web Graph every month from its regular crawl of the web. The builder took three consecutive monthly releases, merged them into a single domain-to-domain link graph of about 121 million domain

nodes connected by 3.9 billion edges (an edge simply means "domain A links to domain B at least once"), and then ran Harmonic Centrality and PageRank calculations across that entire graph offline, as a batch job. Nothing on the site is scraped live; every number displayed is a lookup into pre-computed results.

For serving, the two tools use two different hosting tricks. The CC Rank Checker keeps twelve historical snapshots (2023 through mid-2026) for the top 10 million domains per period in Cloudflare KV, a globally distributed key-value store, sitting behind a Cloudflare Worker. That combination is what makes typing a domain and getting an instant answer possible, essentially the same pattern a CDN uses to serve a static file instantly from the nearest data center. The heavier Referring Domains Checker, which has to list potentially thousands of referring domains and run the Link Gap comparison logic, is a thin Cloudflare Worker acting as a proxy in front of a Python backend hosted on a Hugging Face Space. Both tools expose plain public JSON endpoints behind the scenes, so the whole

thing could be scripted or embedded elsewhere without touching the front-end at all.

Because building the graph is a heavy offline computation rather than a live query, freshness is not continuous. Moving to a newer Common Crawl release means re-downloading the raw files, rebuilding the merged graph, recomputing both ranking algorithms, and re-uploading the results to Cloudflare KV and the Hugging Face backend, a manual rebuild-and-redeploy cycle rather than an always-current index.

5. Why This Matters for SEO: Real Use Cases

In practice, a website owner would use a tool like this in four everyday situations. Before a link-building campaign, checking a prospective partner site's Authority Score answers whether a guest post or partnership is actually worth the effort. When investigating why a competitor outranks you for a keyword despite thinner content, the Link Gap Finder surfaces the specific domains that link to them but not to you, turning a vague feeling of "they must have better SEO"

into a concrete outreach list. Tracking your own domain's HC Rank and PageRank across the twelve historical snapshots shows whether a recent PR push, guest post, or listing genuinely moved the needle over time. Finally, before spending money on a paid backlink audit tool, a free directional check like this one can confirm whether a deeper paid investment is warranted at all.

6. Honest Limitations, Straight From the Tool's Own Documentation

The tool's own FAQ is refreshingly candid about what its numbers do not capture, and any rebuild should inherit the same caveats. Referring domain counts are always lower than what a paid tool like Ahrefs would report, because Common Crawl de-duplicates at the domain level, so a site linking to you from ten thousand different pages still counts as one referring domain. The graph only reflects three specific monthly snapshots rather than a rolling multi-year archive, so a link that existed briefly or was only crawled in a different month simply will not appear. Common Crawl's

main fetcher reads static HTML and does not execute JavaScript, so links injected client-side through widgets, review badges, or single-page app navigation are invisible even though a real browser would render them fine. Pages blocked by robots.txt are skipped entirely, and Common Crawl's own crawl budget and seed list mean long-tail or very new pages may not be selected for any given month at all. The builder's own guidance is to treat a low number as "this many links were visible in the snapshots we indexed," not as proof that a site has few real backlinks.

7. How This Compares to Paid Tools

Ahrefs, Semrush, Moz, and Majestic all run their own continuous, dedicated crawlers that fetch billions of URLs specifically to discover links, and they keep years of historical data merged together, which is why their referring-domain and backlink counts are almost always higher and fresher than Common Crawl's. Common Crawl is a general-purpose research crawl, not a specialized backlink index, so its coverage of any one site is best treated as a large but

incomplete free sample rather than a one-to-one replacement for a paid index. The tradeoff is entirely reasonable: this project gives up completeness and real-time freshness in exchange for being free, open, and instantly rebuildable by anyone, which is exactly why it is worth studying as a blueprint.

8. Blueprint: Building Something Similar for Your Own Site

Path A: Rebuild it the same way, on free Common Crawl data

Download the Host-level or Domain-level Web Graph files for one or more monthly Common Crawl releases from commoncrawl.org or its AWS Open Data bucket, both free.

Parse the edge list into a graph structure using a library built for graphs at this scale, such as graph-tool, igraph, NetworkX for smaller experiments, or a Spark/GraphFrames job if you want it to scale past a single machine.

Compute PageRank and Harmonic Centrality (or a simpler proxy like eigenvector centrality or plain in-degree if the

compute is limited) across the merged graph. Rescale the raw ranks to a friendly 0-100 score the way this tool does.

Load the results into a fast key-value store such as Cloudflare KV, Redis, or DynamoDB for instant single-domain lookups, and build a small API and front end on top, exactly mirroring the two-tool structure described above.

Isn't Common Crawl already free? Yes — here is exactly what that does and does not buy you.

Yes, genuinely free: the Common Crawl Web Graph data itself costs nothing. It is hosted on Amazon's Open Data program, so there are no download fees, no API keys, and no usage limits on the raw files.

What is not free is the engineering to turn those raw files into a working tool: the domain-level graph alone is around 121 million domains and 3.9 billion links, and the full edge file for a single monthly release is roughly 13 to 14 GB compressed. You need somewhere to store that, a process to join it into a lookup table, and a server to answer searches instantly. That

part is what Path B (DataForSEO) sells you pre-built. See Section 12 below for a concrete, automatable way to use the free data directly.

Path B: Skip the graph theory, use DataForSEO instead

Since you already have DataForSEO access, this is the realistic fast path. DataForSEO's Backlinks API already exposes a Backlinks endpoint, a Backlink Summary endpoint, a Referring Domains style endpoint, and a Links Intersection endpoint that does almost exactly what the Link Gap Finder does: it returns domains linking to your competitors but not to you.

DataForSEO also publishes its own transparent Link Rank scoring, similar in spirit to the Authority Score shown on webgraph.metehan.ai. Rather than downloading raw web-graph files and running your own PageRank computation, you would call these endpoints for your own domain and a handful of competitors, store the results, and build the same front-end experience: a search box, a score card, a referring domains table, and a gap-analysis view.

This removes essentially all of the graph-processing and infrastructure work described in Path A, at the cost of an ongoing per-lookup fee instead of a one-time compute cost.

9. Budget Estimate

There are two separate cost centers to budget for: the data or API you rely on for link intelligence, and the engineering plus hosting needed to turn that data into a usable tool. The tables below separate these so you can mix and match, for example using DataForSEO for data while hosting the front end yourself.

9.1 Data and API Cost Comparison

Provider	Cost Model	What You Get
Common Crawl WebGraph	Free public dataset; you only pay for your own compute and	Raw monthly host and domain-level link graph; no ready-made API or

	storage	UI, you build both
DataForSEO Backlinks API	Pay-as-you-go: about $0.024 per request plus $0.000036 per row (roughly $0.06 for a 1,000-row report); $50 minimum account top-up	Backlinks, Backlink Summary, Referring Domains, and a Links Intersection endpoint that mirrors the Link Gap Finder; results in about 2 seconds
Moz API	Monthly subscription tiers: Starter $20-$75/mo (3,000-15,000 rows), Growth $125-$500/mo (50,000-500,000	Linking root domains, Link Intersect, Brand Authority, plus keyword data bundled into the same plans

	rows), Advanced $2,000/mo (4 million rows), Enterprise $10,000/mo (40 million rows)	
Majestic API	Monthly subscription: $399.99/mo API plan (100 million analysis units); Lite $49.99/mo and Pro $99.99/mo exist but do not include full API access	Trust Flow and Citation Flow (its own authority metrics), referring domains, referring IPs, and historic index data
Ahrefs API	Historically bundled with	Backlinks, referring domains,

	higher-tier Ahrefs plans plus metered API credits rather than a simple self-serve rate; check ahrefs.com/api directly for current terms, since access could not be verified live during this research	and Domain Rating, widely regarded as one of the largest independent crawl indexes

9.2 One-Time Build Cost (Path A: self-hosted Common Crawl clone)

These are one-off costs to get a working clone live for the first time.

Cost Item	Estimate	Notes
Data acquisition	$0	Common Crawl WebGraph files are free to download from the AWS Open Data bucket; only minor bandwidth cost if pulled outside AWS
One-off data sync & join pipeline	$20-$100	Common Crawl already precomputes Harmonic Centrality and PageRank for every release

		(domain-ranks.txt.gz, ~2.2GB). You only need to download that file plus domain-vertices.txt.gz (~858MB), then stream-join them by ID on a small instance; no high-memory VM or custom ranking computation is required. The 13.6GB domain-edges.txt.gz is only needed if you want to replicate Link Gap

		Finder-style edge queries yourself.
Engineering time (if you hire it out)	$2,500-$12,000	A freelance data or backend engineer plus a front-end developer to build the sync/join pipeline, API, and UI. This is cheaper than a typical graph-computation build since Common Crawl already supplies precomputed ranks; wide range depending on scope, region, and

		whether you build one snapshot or twelve.

9.3 Ongoing Monthly Cost, Path A (self-hosted)

Cloudflare Workers stays free for most small-to-medium traffic, with paid tiers starting around $5 per month once you exceed free-tier request limits. Cloudflare KV is free for roughly 100,000 reads per day and then costs about $0.50 per million reads plus a small storage fee. A Hugging Face Space can run on a free CPU tier for light use, or $9 to $25 or more per month for faster, persistent hardware if the Referring Domains and Link Gap queries get heavy traffic. If you want monthly refreshes to match Common Crawl's release cadence, budget only about $5 to $20 per month for the small sync job that downloads and joins the new precomputed rank files, since no heavy ranking computation is needed on your end. All together, a lightly used clone realistically runs $10 to

$40 per month, rising to $50 to $150 per month if traffic grows or you also ingest the full edges file for Link Gap Finder parity.

9.4 Ongoing Monthly Cost, Path B (DataForSEO-powered)

This route has no graph-processing infrastructure at all, so nearly the entire budget is the DataForSEO bill itself, plus a few dollars a month for a small serverless function to host your own thin front end. You first add the required $50 minimum top-up to activate pay-as-you-go billing. For light internal use, such as checking your own domain plus five to ten competitors once a month, expect roughly $5 to $20 per month. For a small public-facing tool with moderate traffic, for example 50 to 200 lookups per day averaging 1,000 to 2,000 rows each, expect roughly $50 to $400 per month, since cost scales directly with rows requested rather than a flat subscription. This is the more predictable, lower-risk starting point given that DataForSEO access already exists.

10. Recommendation

Given that DataForSEO access is already in hand, the fastest and lowest-risk way to get a similar tool running for your own website is Path B: call the Backlinks, Backlink Summary, and Links Intersection endpoints for your domain and a short list of competitors, rescale the results into a friendly 0-100 score the way this tool does, and wrap it in a simple search box and results page. This can realistically be a days-to-weeks project rather than months, for well under a few hundred dollars a month even with real traffic. Building your own Common Crawl-based graph, Path A, only makes sense later if you want a completely free, publicly embeddable tool at large scale, want independence from any paid vendor, or want the exercise of learning graph algorithms firsthand. Many teams start with Path B to validate that people actually want the tool, then consider Path A once usage justifies the extra engineering investment.

11. Ready-to-Use Replit Build Prompt

Copy everything in the box below and paste it directly into Replit's AI Agent (Replit Agent) to scaffold Path B of this tool — a DataForSEO-powered domain authority checker. It assumes you will add your own DataForSEO login/API key as a Replit Secret.

```
Build a web app called "Domain Authority Explorer" - a domain backlink and authority checker similar to webgraph.metehan.ai, powered by the DataForSEO API.

Tech stack: Node.js + Express backend, simple HTML/CSS/JavaScript frontend (or React), deployed as a single Replit app. Store the DataForSEO login and password as Replit Secrets named DATAFORSEO_LOGIN and DATAFORSEO_PASSWORD. Never hard-code
```

credentials, and never expose them to the browser - all DataForSEO calls must happen server-side only.

Core features to build:

1) A homepage with one search box where a user types a domain (example.com) and clicks Check.

2) On submit, call the DataForSEO Backlinks Summary endpoint (POST https://api.dataforseo.com/v3/backlinks/summary/live) for that domain using HTTP Basic Auth with the Secrets.

3) Display total backlinks, referring domains, referring main domains, and broken backlinks. Normalize DataForSEO's domain rank metric into a 0-100 Authority Score, clearly labeled as DataForSEO's own rank, not Google PageRank.

4) Add a panel showing the top 20

referring domains using the Backlinks endpoint (POST /v3/backlinks/backlinks/live), filtered to one link per target domain, sorted by referring domain rank descending.

5) Add a Top TLDs bar chart, grouping the referring domains client-side by top-level domain (.com, .org, .io, etc.) - no extra API call needed.

6) Add a Link Gap Finder page: inputs for Your Domain and up to 5 comma-separated Competitor Domains. Call the DataForSEO Domain Intersection endpoint (POST /v3/backlinks/domain_intersection/live) to find domains linking to competitors but not to your domain, sorted by referring domain rank, with a Platform Opportunity badge for domains with unusually high outbound link counts.

7) Cache every API response per domain for 24 hours (Replit Database or a local SQLite/JSON cache) so repeat lookups do not re-spend API credits.

8) Show a running estimated cost so far this session, based on DataForSEO pay-as-you-go pricing (about $0.024 per API call plus a small per-row cost), so spend stays visible.

9) Handle errors gracefully: if DataForSEO returns no data or a call fails or times out, show a friendly message instead of crashing.

10) Keep the UI minimal: a search bar, score cards at the top, a sortable results table, and a lightweight chart library (such as Chart.js) for the TLD bar chart.

Do not log or display the DataForSEO credentials anywhere in the UI, console,

or client-side code.

DIFFERENTIATION FEATURES (add after the core build works):

11) Historical trend: each time a domain is looked up, store the date, Authority Score, and referring domain count in a lookups_history table, and show a simple line chart of past checks for that domain.

12) Bulk compare: accept up to 5 comma-separated domains at once (already partly supported by the Link Gap Finder input) and render them in one sortable comparison table instead of one at a time.

13) Spam flag: for each referring domain returned, flag it as “low diversity” if DataForSEO’s data shows very few distinct outbound target domains, surfaced as a small warning icon next to suspicious entries.

14) Niche tag (optional stretch goal): let the user manually tag a lookup with an industry label, and show an "authority within tagged niche" percentile alongside the raw global score, computed from the user's own saved lookup history.

12. How to Actually Use the Common Crawl Web Graph, Automated

Here is the good news the FAQ undersells: Common Crawl does not just hand you raw links, it already computes Harmonic Centrality and PageRank for every domain and publishes the finished ranks as a downloadable file with every monthly release. You do not need to run your own PageRank job unless you want a custom subgraph. Verified directly on data.commoncrawl.org, each release (for example cc-main-2026-apr-may-jun) publishes, per domain, three key files: a vertices file (domain name plus a numeric ID), an

edges file (which domain links to which, by ID), and a ranks file containing harmonic centrality and pagerank already calculated. The domain-level files for one release are about 860 MB (vertices), 2.2 GB (ranks), and 13 to 14 GB (edges) compressed - all free to download, hosted on AWS's Open Data program with no egress fees.

A practical, automatable pipeline looks like this:

1) Check for new releases automatically by fetching the Web Graphs Index (index.commoncrawl.org/web-graphs-index.html) or its JSON feed, and comparing the newest release name to the last one you processed.

2) Download only the two small files, not the huge edges file, unless you specifically need referring-domains lists: domain-vertices.txt.gz and domain-ranks.txt.gz from data.commoncrawl.org/projects/hyperlinkgraph/<release-name>/.

3) Stream-join the two files by their numeric ID. Both files are sorted by ID, so you can read them line by line together without loading either into memory, producing rows of (domain, harmonic centrality, pagerank).

4) Normalize both scores into your 0-100 Authority Score and Citation Score, then load everything into a lightweight database such as SQLite or DuckDB, indexed by domain for instant lookups.

5) Swap the new table in atomically (build it under a new name, then point the live app at it) so the site never serves a half-loaded dataset and has zero downtime during monthly refreshes.

6) Only if you want the Referring Domains or Link Gap Finder features, download the much larger domain-edges.txt.gz too, and instead of loading it whole, query it on demand for just the domain(s) a user searches, or load it into DuckDB, which can filter compressed files efficiently without a full custom index.

This whole pipeline is a perfect fit for a scheduled job (a monthly cron job, a GitHub Actions workflow, or a Replit Scheduled Deployment) since Common Crawl only republishes the graph roughly once a month anyway - there is no benefit to running it more often.

12.1 Ready-to-Use Automation Prompt (Common Crawl Sync Job)

Paste this into Replit's AI Agent (or use it as a spec for any scheduled scripting environment) to build the free-data ingestion pipeline described above.

```
Build a scheduled job called "Common Crawl Graph Sync" that keeps a local domain-authority lookup table up to date using Common Crawl's free, precomputed Web Graph releases.

Tech stack: Python 3 (requests, gzip, csv), DuckDB or SQLite as the local
```

database, deployed on Replit using a Scheduled Deployment that runs once a month. No API keys or secrets are needed since all Common Crawl data is public.

1) Fetch the release list from index.commoncrawl.org/web-graphs-index.html (or its JSON feed) and read the newest release name, such as cc-main-2026-apr-may-jun.

2) Compare it to a stored last_processed_release value (kept in a small state file or a table). If they match, log 'already up to date' and exit without downloading anything.

3) If it is a new release, stream-download these two files from https://data.commoncrawl.org/projects/hyperlinkgraph/<release-name>/: <release-name>-domain-vertices.txt.gz and

<release-name>-domain-ranks.txt.gz. Do not download the -domain-edges.txt.gz file in this basic version, since it is tens of gigabytes and only needed for referring-domains features.

4) Decompress both files as streams (do not load fully into memory) and read them line by line in parallel, since both are sorted by the same numeric ID. Join them into rows of (domain, harmonic_centrality, pagerank).

5) Convert the reversed domain notation used by Common Crawl back into normal domain order, and normalize harmonic_centrality and pagerank into 0-100 percentile scores.

6) Write the results into a new database table named after the release (for example domains_cc_main_2026_apr_may_jun), then

atomically update a current_table pointer so the live app always reads a fully-loaded table and never sees partial data.

7) Delete or archive the previous release's table once the new one is confirmed working, to save disk space.

8) Update last_processed_release, and log how many domains were loaded and how long the job took.

9) Add retry-with-backoff around the file downloads (these are multi-gigabyte transfers that can fail partway), and make the whole job safe to re-run from scratch if it fails partway through.

10) Expose one function, get_domain_authority(domain), that looks up a domain in the current table and returns its Authority Score and Citation

```
Score, so the main lookup web app (built separately) can call it directly.
```

12.2 Ready-to-Use Replit Build Prompt (Path A — Full Common Crawl-Powered Tool)

Copy everything in the box below and paste it directly into Replit's AI Agent to scaffold a complete Path A tool — a Common Crawl-powered domain authority checker with its own sync job, database, API, and front end. No third-party API keys are required since Common Crawl's data is public.

```
Build a web app called "Domain Authority Explorer" - a domain backlink and authority checker similar to webgraph.metehan.ai, powered entirely by Common Crawl's free precomputed Web Graph instead of a paid API.

Tech stack: Python 3 (FastAPI) backend
```

plus a lightweight HTML/CSS/JavaScript frontend, deployed as a single Replit app. Use DuckDB or SQLite as the local database. No API keys or secrets are needed since all Common Crawl data is public.

PART 1 - Data sync job (run on a Replit Scheduled Deployment once a month):

1) Fetch the release list from index.commoncrawl.org/web-graphs-index.html (or its JSON feed) and read the newest release name, such as cc-main-2026-apr-may-jun.

2) Compare it to a stored last_processed_release value. If they match, log “already up to date” and exit without downloading anything.

3) If it is a new release, stream-download <release-name>-domain-vertices.txt.gz and

<release-name>-domain-ranks.txt.gz from https://data.commoncrawl.org/projects/hyperlinkgraph/<release-name>/. Skip the much larger -domain-edges.txt.gz file in this basic version.

4) Decompress both files as streams (do not load fully into memory) and join them by their shared numeric ID into rows of (domain, harmonic_centrality, pagerank).

5) Convert the reversed domain notation Common Crawl uses back into normal domain order, and normalize harmonic_centrality and pagerank into 0-100 percentile scores, labeled “Common Crawl Authority Score” and “Common Crawl Popularity Score” - never call it Google PageRank.

6) Write results into a new table named after the release, then atomically swap a current_table pointer once loading is

confirmed complete, so the live app never serves a half-loaded dataset. Archive the previous release's table afterward.

7) Log how many domains were loaded and how long the job took, update last_processed_release, and wrap downloads in retry-with-backoff so partial failures can safely re-run.

PART 2 - Backend API:

8) Expose GET /api/domain/<domain> returning that domain's Authority Score, Popularity Score, and the Common Crawl release date the data came from; return a clear "not found in latest Common Crawl crawl" message for unknown domains.

9) Expose GET /api/status returning the current release name and when it was last synced, so the frontend can show data freshness.

PART 3 - Frontend:

10) Build a homepage with one search box where a user types a domain (example.com) and clicks Check.

11) Display the Authority Score and Popularity Score as large, clearly labeled numbers, plus a small note showing the data source and release date.

12) Add a footnote clarifying these are Common Crawl's own harmonic centrality and PageRank-style metrics, not Google's, and that the data refreshes roughly monthly.

Keep the whole app in one Replit project, and make sure the sync job and the web server can run independently of each other.

DIFFERENTIATION FEATURES (add after the core build works):

13) Historical trend: when writing each new release's table, also append that domain's score into a lookups_history table (domain, release_name, authority_score) whenever a user has previously looked it up, and plot a simple trend line across releases.

14) New and lost referring domains (needs the edges file): if you also ingest the domain-edges.txt.gz for a domain of interest, diff this release's inbound edges against last release's and list "new referring domains this month" and "referring domains lost this month".

15) Bulk compare: accept up to 5 comma-separated domains in one request and return them as one ranked table instead of one at a time.

16) Spam flag heuristic: when showing

referring domains (from the edges file), flag any referring domain with very low outbound diversity as a possible low-quality or PBN-style link. 17) Packaging (stretch goal): expose the same get_domain_authority(domain) function as a small public read-only API endpoint, so the tool can later power a Chrome extension or Google Sheets add-on.

13. Path A vs Path B: Pros and Cons

A side-by-side view of the same trade-off discussed throughout this document:

Path A: Rebuild on Common Crawl	**Path B: Use DataForSEO**

Pros: Data acquisition is genuinely free with no per-lookup cost or rate limits. Ranking methodology is transparent and published (the WebGraph framework). Once built, it can be embedded or redistributed at unlimited scale. Doubles as real experience in large-scale graph engineering.	Pros: Can be built and working within a day since crawling and ranking are already done. Minimal infrastructure to build or maintain. Backed by DataForSEO's own crawler plus ready-made extra endpoints (anchors, history, domain intersection). Support and reliability handled by a commercial vendor.
Cons: Real engineering time to build, typically days to weeks. Multi-gigabyte monthly downloads and processing to maintain.	Cons: Ongoing per-request cost that scales with usage, never fully free. Dependent on a third-party vendor for pricing, rate limits, and

Freshness limited to Common Crawl's roughly monthly release cadence. Referring-domains and Link Gap features need the much larger edges file and more complex querying. You own all future maintenance and bugs.	terms. Ranking methodology is proprietary rather than open. Requires an active DataForSEO account and credentials.

14. Ideas for Differentiation and Originality

The base rebuild (Path A or Path B) gets you to feature parity with webgraph.metehan.ai. The ideas below are ways to go beyond parity and build something genuinely differentiated, grounded in gaps found during this research (see Section 6, Honest Limitations).

1. Hybrid score. Blend Common Crawl's free, transparent structural authority with DataForSEO's fresher backlink and

anchor-text data into one composite score, clearly labeling which parts come from which source. Neither the original tool nor a single-path clone offers this.

2. Historical trend lines. The original tool only shows a single snapshot. Since Common Crawl republishes monthly, store every release's score per domain looked up and plot a simple trend line showing whether authority is rising or falling over time.

3. Bulk and comparison workflows. Let a user paste in several competitor domains, or upload a CSV, and see them ranked side by side, shifting the tool from a single lookup utility toward competitive research.

4. New and lost referring domain alerts. The monthly sync job already diffs one release against the last. Surfacing that diff as "new domains linking to you this month" and "domains that dropped off" turns the tool from a static lookup into an ongoing monitoring feature.

5. Topical or niche-relative scoring. A domain with modest global authority can be dominant within a narrow niche. Layering in a lightweight topical tag and showing authority relative to that niche, alongside the raw global score, is something neither the original tool nor typical DR-style metrics do cleanly.

6. Spam and toxicity flags. A simple heuristic pass over referring domains, flagging low outbound diversity or obvious PBN-style patterns, turns the referring-domains list into a disavow-prep tool rather than a plain list.

7. Packaging and distribution. A Chrome extension or Google Sheets add-on that surfaces the score inline while browsing or prospecting is a distribution advantage the original web-only tool does not offer.